Feature Model Configuration as SAT: Group Project Report, Part II

Ana Carvalhal (up202305274), André Sousa (up202109775),
João Taveira (up202306063), Rodrigo Rocha (up202305891)

I. Introduction

For the Advanced Software Construction Techniques (TACS) class project, our group was assigned the B theme: Search with Logical Constraints, specifically the SAT problem, applied to product configuration from a feature model. In the first part we designed the two languages of our chain, the Problem DSL and the Solution DSL, with their OCL constraints. This report covers the second part of the project, which includes the model-to-model transformations T1 and T2, the use of BESSER's Qiskit generator (T3), the execution on the Aer simulator and the validation against a classical baseline. This report will document how a feature model written in our Problem DSL is transformed into a CNF formula and then into a Grover circuit, which references each transformation follows, the results we obtained for our five test instances and the limitations of our approach. The whole chain runs with one command, `python run_all.py`, and no generated file is edited by hand.

II. Changes to Part I of the Work

As a reminder, Part I designed a Problem DSL for feature models and a Solution DSL for CNF formulas, with one hand-built example. The changes came mainly from the teachers' feedback and from running our models with BESSER for the first time:

- The main example grew from 7 to 11 features (with a mandatory `battery`, and `temperature`, `steps` and `weather` under `digital`), as requested by the teachers;
- There are now five test instances, all from the SmartWatch family, instead of a single example; they are described in `Part1/Instances.md`;
- The `parent`/`children` links were corrected: BESSER had written them as self-links, and we only noticed when our `NoCycles` check failed;
- The OCL invariants are also Python functions with the same names in `checks.py`, since the BESSER OCL evaluator could not evaluate most of our constraints.

III. Model-to-Model Transformations

Following the reference chain given by the teachers, both transformations are Python functions over BESSER object models. Our main concern was that every rule could be traced to the literature, so each rule in the code has a comment with the reference it implements.

TABLE I
Rules of T1 (P parent, C child, M_i group members)

	Source element	Clause(s)	Ref.
R1	root	$(root)$	[1], [2]
R2	mandatory feature	$(\neg P \vee C)$	[1], [2]
R3	any non-root feature	$(\neg C \vee P)$	[1], [2]
R4	`Or`/`Xor` group	$(\neg P \vee M_1 \vee \cdots \vee M_k)$	[1], [2]
R5	`Xor` group	$(\neg M_i \vee \neg M_j)$, all $i < j$	[1], [2]
R6	A requires B	$(\neg A \vee B)$	[1]
R7	A excludes B	$(\neg A \vee \neg B)$	[1]
R8	`FeatureChoice`	(X) or $(\neg X)$	[1]

A. T1: Problem DSL to Solution DSL

T1 (`t1_problem_to_cnf.py`) takes a `FeatureModel` and one `SelectionRequest` and builds a `CnfFormula`. Each feature becomes one `Variable` [1]. The variable `index` (the qubit position) is a depth-first pre-order of the feature tree, children sorted by id, so the result is deterministic. The clauses come from the eight rules of Table I. T1 also returns a trace (rule, source element, reference per clause) that `run_t1.py` writes to `generated/T1_report.md`.

The solver choices are not fixed in the transformation. `shots` is a parameter of T1, and the number of Grover rounds is computed as $k = \lfloor \frac{\pi}{4}\sqrt{N/M} \rfloor$ with $N = 2^n$ [3], where M is the number of solutions counted by the classical baseline. Both values, together with `knownSolutions` $= M$, are stored in the `SolverConfig` of the output model, and T2 only reads them from there. T1 checks the Part I invariants on its input and on its output (`checks.py`) and stops if any of them fails.

B. T2: Solution DSL to Quantum Circuit

T2 (`t2_cnf_to_quantum.py`) builds an instance of BESSER's quantum metamodel. The circuit has n search qubits (qubit i is the `Variable` with index i), a counter register and one output qubit. It follows the standard Grover structure [4], [5]: Hadamard on the search qubits, output qubit in $|-\rangle$ for phase kickback, then `rounds` repetitions of oracle and diffuser, and finally the measurement of the search qubits. Oracle and diffuser are sub-circuits (`FunctionGate`) defined once and applied k times.

The oracle of the course slides [7] uses one ancilla per clause ($n+m+1$ qubits); with that construction only I1 stays below the 20 qubit limit (I4 would need 35). We use instead a binary counter of violated clauses [6]: for each clause with two

TABLE II
VALIDATION AGAINST THE BRUTE-FORCE BASELINE (VALUES VARY SLIGHTLY BETWEEN RUNS DUE TO SAMPLING NOISE)

	n	m	qubits	M	k	P_{meas}	P_{theo}	found
I1	4	8	8	1	3	0.981	0.961	1 of 1
I2	7	15	12	1	8	0.995	0.996	1 of 1
I3	11	20	17	8	12	1.000	1.000	8 of 8
I4	11	23	17	1	35	1.000	1.000	1 of 1
I5	11	20	17	0	1	0.000	0.000	0 of 0

or more literals, a multi-controlled increment adds 1 when the clause is false. A clause is false for exactly one combination of its variables, so the control pattern is an anti-control for each positive literal and a control for each negated one. The output qubit is flipped when the counter is 0, and the same gates in reverse order uncompute the counter. Unit clauses (from R1 and R8) need no ancilla: the search qubit itself controls (or anti-controls) the marking gate [5]. With K clauses with two or more literals ($K \leq m$), the counter needs $\lceil \log_2(K+1) \rceil$ qubits, so the largest instances fit in 17 qubits. The diffuser is the usual inversion about the mean [4], [5], reusing the output qubit for the phase flip of $|0 \ldots 0\rangle$.

IV. MODEL TO TEXT

For the model-to-text step we did not write a generator of our own. T3 is BESSER's `QiskitGenerator`, called with the circuit from T2 and the `shots` of the `SolverConfig`. The generated script is executed unchanged. Decoding is also derived from the models: the bit of `Variable` i is character $-1-i$ of each measured bitstring (Qiskit is little-endian), and the feature names come from the Problem DSL model. So a measurement such as `1011` in I1 is reported as *Digital, Display, Watch*.

V. VALIDATION

The classical baseline (`baseline.py`) is a brute force over the 2^n assignments of the CNF. For each instance `validation.py` compares the measured counts with it, and reports the success probability (share of shots that are valid configurations) next to the theoretical value $\sin^2((2k+1)\theta)$, with $\sin^2\theta = M/N$ [3]. The instances are the SmartWatch family of Part I and cover the kinds asked in Annex A: tiny (I1), unique solution (I2, I4), several solutions (I3) and no solution (I5). Table II shows the results with 1024 shots.

In every instance all measured valid configurations are baseline solutions, and the most frequent bitstring is one of them. In I3 the eight solutions appear with similar frequency. In I5 (analog and GPS together, impossible because GPS requires digital) no valid configuration is measured, as expected. Grover calls the oracle k times per shot (35 in I4) while the brute force evaluates the formula $N = 2048$ times, the expected $\sqrt{N}$ behaviour. Unit tests (`test_part2.py`) check that the CNF from T1 has the same solutions as the feature model semantics, and that the oracle marks exactly the baseline solutions. The full table and the answer in domain terms are regenerated by `run_all.py` in `generated/results.md`. This is the back-mapping from the measured bitstring to the Problem DSL: bit i is Variable i, whose name is a feature id (e.g., *"a valid product: Watch, Display, Digital"*).

VI. LIMITATIONS

Despite the successful end-to-end execution of our chain, several limitations remain.

Prior knowledge of M. The Grover round formula requires M, which T1 obtains by classical brute-force evaluation of the CNF. In a real quantum-advantage scenario, M is unknown and would need quantum counting or an iterative strategy with growing rounds [3]. For $M = 0$ (e.g., I5), we set rounds to 1 (the minimum allowed by our OCL constraints) and rely on classical post-analysis.

Circuit depth and hardware noise. The binary counter reduces the qubit footprint but increases gate depth: each oracle call uses $2Kc$ multi-controlled gates, some with large control sets. Since we target an ideal simulator (Qiskit Aer), deep circuits such as I4's 35 rounds would require error mitigation on near-term NISQ hardware.

Expressiveness of the Problem DSL. The DSL supports mandatory and optional features, `Or` and `Xor` groups, and binary requires/excludes constraints. Arbitrary propositional cross-tree constraints, feature attributes and numerical cardinalities are unsupported. The pairwise encoding of `Xor` groups (Rule R5) also scales quadratically with group size.

Domain decoding and feedback loop. Decoding into domain terms is done by the validation module (derived from the models), not inside the generated Qiskit script; the generated script is executed unchanged. The answer in domain terms is written to `generated/results.md`. Valid configurations are not written back as an instance model of the Problem DSL.

Scalability. Constrained by the qubit limit (≤ 12 variables) and by the exponential cost of classical validation, our instances stay small (up to 11 variables and 17 qubits), where classical solvers are inherently faster than simulation.

VII. USE OF LLMS

LLMs assisted with BESSER's API and with reviewing the transformations; every output was checked against the literature and the baseline. The full log is in `AILog.md`.

REFERENCES

[1] D. Batory, "Feature models, grammars, and propositional formulas," in *Proc. SPLC*, LNCS 3714, Springer, 2005, pp. 7–20.

[2] S. Thaker, D. Batory, D. Kitchin, and W. Cook, "Safe composition of product lines," in *Proc. GPCE*, 2007, pp. 95–104.

[3] M. Boyer, G. Brassard, P. Høyer, and A. Tapp, "Tight bounds on quantum searching," *Fortschritte der Physik*, vol. 46, pp. 493–505, 1998.

[4] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. STOC*, 1996, pp. 212–219.

[5] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th anniversary ed. Cambridge Univ. Press, 2010, sec. 4.3 and 6.1.

[6] A. Alasow, P. Jin, and M. Perkowski, "Quantum algorithm for variant maximum satisfiability," *Entropy*, vol. 24, no. 11, 1615, 2022.

[7] J. Pascoal Faria and J. Cunha, "Quantum computing: a brief introduction," TACS 2026/27 course slides, FEUP, 2026.